

Integrantes

Hanna Paola Ramírez Casillas

Jael Sadai Cabrera Gómez

Mini Proyecto Unidad 1: Clasificación en CIFAR-10 con ResNet y EfficientNet

- Pipeline completo de clasificación con CNN (ResNet o EfficientNet) sobre CIFAR-10 (Partes 1 a 5)
- Comparativa técnica de dos arquitecturas CNN con curvas de aprendizaje y métricas (Partes 5 y 6) *Reporte de diseño (arquitectura, hiperparámetros, estrategia de entrenamiento) (Parte 7)

Recomendación de entorno

Activen GPU en Colab: **Entorno de ejecución → Cambiar tipo de entorno de ejecución → GPU**. ResNet-18 y EfficientNet-B0 a resolución 224×224 son considerablemente más pesados que la CNN pequeña de la práctica anterior; en CPU van a funcionar, pero mucho más lento. Las constantes `TAM_TRAIN`, `TAM_VAL` y `N_EPOCAS` están pensadas para ajustarse según el tiempo y el hardware disponible.

0. Configuración inicial

```
!pip install -q torchmetrics
```

```
import time

import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.models as models
import torchvision.transforms as transforms
from torch.utils.data import DataLoader, random_split
import matplotlib.pyplot as plt
```

```
import pandas as pd

from torchmetrics.classification import (
    MulticlassAccuracy,
    MulticlassPrecision,
    MulticlassRecall,
    MulticlassF1Score,
    MulticlassConfusionMatrix,
)

torch.manual_seed(42)

DISPOSITIVO = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Dispositivo:", DISPOSITIVO)

Dispositivo: cuda
```

✓ 1. Datos: CIFAR-10 a 224x224

ResNet y EfficientNet, tal como se definen en `torchvision.models`, esperan entradas de 224x224 pixeles (el tamaño estándar de ImageNet en el que se diseñaron y evaluaron originalmente). Redimensionamos CIFAR-10 (32x32 nativo) a ese tamaño.

```
MEDIA_IMAGENET = (0.485, 0.456, 0.406)
STD_IMAGENET = (0.229, 0.224, 0.225)

transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(MEDIA_IMAGENET, STD_IMAGENET),
])

# COMPLETAR CARGAR IMAGENES DE ENTRENAMIENTO Y PRUEBA Y GENERAR UN SUBCONJUNTO
trainset_completo = torchvision.datasets.CIFAR10(root='./data', train=True, download=True)
testset = torchvision.datasets.CIFAR10(root='./data', train=False, download=True)

tam_train = int(0.8 * len(trainset_completo))
tam_val = len(trainset_completo) - tam_train
train_subset, val_subset = random_split(trainset_completo, [tam_train, tam_val])

nombres_clases = trainset_completo.classes
print("Clases:", nombres_clases)
print("Tamaño del set de entrenamiento completo:", len(trainset_completo))
print("Tamaño del set de prueba:", len(testset))

Clases: ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse']
Tamaño del set de entrenamiento completo: 50000
```

Tamano del set de prueba: 10000

✓ 1.1 Subconjuntos de trabajo

```
BATCH_SIZE = 32
train_loader = DataLoader(train_subset, batch_size=BATCH_SIZE, shuffle=True, num_workers=2)
val_loader = DataLoader(val_subset, batch_size=BATCH_SIZE, shuffle=False, num_workers=2)
test_loader = DataLoader(testset, batch_size=64, shuffle=False, num_workers=2)

print(f"Entrenamiento: {len(train_subset)} imagenes | Validacion: {len(val_subset)} imagenes")

def desnormalizar(img_tensor):
    """Invierte la normalizacion para poder visualizar la imagen."""
    media = torch.tensor(MEDIA_IMAGENET).view(3, 1, 1)
    std = torch.tensor(STD_IMAGENET).view(3, 1, 1)
    return (img_tensor * std + media).clamp(0, 1)

imagenes, etiquetas = next(iter(train_loader))
fig, axs = plt.subplots(1, 6, figsize=(12, 2.5))
for i, ax in enumerate(axs):
    ax.imshow(desnormalizar(imagenes[i]).permute(1, 2, 0))
    ax.set_title(nombres_clases[etiquetas[i]], fontsize=10)
    ax.axis("off")
plt.suptitle("Ejemplos del subconjunto de entrenamiento (redimensionados a 224x224)")
plt.show()
```

Entrenamiento: 40000 imagenes | Validacion: 10000 | Prueba: 10000

Ejemplos del subconjunto de entrenamiento (redimensionados a 224x224)



✓ 2. Construir ResNet-18 y EfficientNet-B0

Tomamos las arquitecturas de `torchvision.models` con pesos preentrenados en ImageNet (`weights="DEFAULT"`) y adaptamos unicamente la ultima capa para que clasifique entre las 10 clases de CIFAR-10 en vez de las 1000 de ImageNet — el resto de la arquitectura, y sus pesos preentrenados, se conservan exactamente como en 1.3.

Ver : <https://docs.pytorch.org/vision/main/models.html>

Ejemplo:

<https://docs.pytorch.org/vision/main/models/generated/torchvision.models.resnet18.html>

```
def construir_resnet18(n_clases=10):  
    """ResNet-18 preentrenada en ImageNet con la ultima capa (fc) adaptada a n_  
    modelo = models.resnet18(weights="DEFAULT")  
    modelo.fc = nn.Linear(modelo.fc.in_features, n_clases)  
    return modelo.to(DISPOSITIVO)
```

```
torch.manual_seed(42)  
modelo_resnet = construir_resnet18(n_clases=10)
```

```
#COMPLETAR construir_efficientnet_b0(n_clases=10)
```

```
def construir_efficientnet_b0(n_clases=10):  
    modelo = models.efficientnet_b0(weights="DEFAULT")  
    modelo.classifier[1] = nn.Linear(  
        modelo.classifier[1].in_features,  
        n_clases  
    )  
    return modelo.to(DISPOSITIVO)
```

```
torch.manual_seed(42)  
modelo_efficientnet = construir_efficientnet_b0(10)
```

✓ 3. Metricas con TorchMetrics

Mismo patron que en la Practica 4 original: `update()` por lote, `compute()` al final de la epoca, `reset()` antes de la siguiente. Usamos `average="macro"` (promedio simple entre las 10 clases)

ver: <https://meta-pytorch.org/torcheval/main/torcheval.metrics.html>

Ejemplo :

"accuracy": MulticlassAccuracy(num_classes=n_clases,
average="macro").to(DISPOSITIVO),

```
def crear_metricas(n_clases=10):
    return {
        "accuracy": MulticlassAccuracy(num_classes=n_clases, average="macro").to(DISPOSITIVO),
        "precision": MulticlassPrecision(num_classes=n_clases, average="macro").to(DISPOSITIVO),
        "recall": MulticlassRecall(num_classes=n_clases, average="macro").to(DISPOSITIVO),
        "f1": MulticlassF1Score(num_classes=n_clases, average="macro").to(DISPOSITIVO)
    }

def correr_epoca(modelo, loader, perdida_fn, metricas, optimizador=None):
    """
    Corre una epoca completa. Si optimizador no es None, entrena (backward + step)
    si es None, solo evalua (torch.no_grad()).
    """
    modo_entrenamiento = optimizador is not None
    modelo.train() if modo_entrenamiento else modelo.eval()

    for m in metricas.values():
        m.reset()

    perdida_acumulada, n_lotes = 0.0, 0
    contexto = torch.enable_grad() if modo_entrenamiento else torch.no_grad()

    with contexto:
        for imagenes, etiquetas in loader:
            imagenes, etiquetas = imagenes.to(DISPOSITIVO), etiquetas.to(DISPOSITIVO)

            if modo_entrenamiento:
                optimizador.zero_grad()

            salida = modelo(imagenes)
            perdida = perdida_fn(salida, etiquetas)

            if modo_entrenamiento:
                perdida.backward()
```

```

        optimizador.step()

    perdida_acumulada += perdida.item()
    n_lotes += 1

    preds = salida.argmax(dim=1)
    for m in metricas.values():
        m.update(preds, etiquetas)

resultados = {nombre: m.compute().item() for nombre, m in metricas.items()}
resultados["loss"] = perdida_acumulada / n_lotes
return resultados

```

✓ 4. Entrenar ambos modelos en igualdad de condiciones

Para que la comparacion sea tecnicamente valida, ambos modelos usan **los mismos datos, el mismo numero de epocas, el mismo optimizador y el mismo learning rate**. Documentamos estos hiperparametros aqui porque los van a necesitar tal cual para el reporte de diseno (Parte 7).

```

# Hiperparametros (identicos para ambos modelos)
N_EPOCAS = 5
LR = 1e-4

def entrenar_modelo(modelo, nombre, n_epocas=N_EPOCAS, lr=LR):
    perdida_fn = nn.CrossEntropyLoss()
    optimizador = optim.Adam(modelo.parameters(), lr=lr)
    metricas_train = crear_metricas()
    metricas_val = crear_metricas()
    historial = {"train": [], "val": [], "tiempo_s": []}

    for epoca in range(1, n_epocas + 1):
        inicio = time.time()

        res_train = correr_epoca(
            modelo, train_loader, perdida_fn, metricas_train, optimizador
        )

        res_val = correr_epoca(
            modelo, val_loader, perdida_fn, metricas_val
        )

        duracion = time.time() - inicio

        historial["train"].append(res_train)
        historial["val"].append(res_val)

```

```

historial["tiempo_s"].append(duracion)

print(
    f"[{nombre}] Epoca {epoca}/{n_epocas} | "
    f"loss train {res_train['loss']:.3f} val {res_val['loss']:.3f} | "
    f"acc train {res_train['accuracy']:.3f} val {res_val['accuracy']:.3f} | "
    f"{duracion:.0f} s"
)

return historial

hist_resnet = entrenar_modelo(modelo_resnet, "ResNet-18")
hist_efficientnet = entrenar_modelo(modelo_efficientnet, "EfficientNet-B0")

```

```

[ResNet-18] Epoca 1/5 | loss train 0.367 val 0.213 | acc train 0.877 val 0.929 |
[ResNet-18] Epoca 2/5 | loss train 0.129 val 0.195 | acc train 0.957 val 0.930 |
[ResNet-18] Epoca 3/5 | loss train 0.069 val 0.202 | acc train 0.978 val 0.936 |
[ResNet-18] Epoca 4/5 | loss train 0.057 val 0.202 | acc train 0.981 val 0.936 |
[ResNet-18] Epoca 5/5 | loss train 0.046 val 0.203 | acc train 0.985 val 0.939 |
[EfficientNet-B0] Epoca 1/5 | loss train 0.450 val 0.143 | acc train 0.861 val 0.929 |
[EfficientNet-B0] Epoca 2/5 | loss train 0.157 val 0.124 | acc train 0.948 val 0.930 |
[EfficientNet-B0] Epoca 3/5 | loss train 0.096 val 0.116 | acc train 0.968 val 0.936 |
[EfficientNet-B0] Epoca 4/5 | loss train 0.066 val 0.117 | acc train 0.978 val 0.936 |
[EfficientNet-B0] Epoca 5/5 | loss train 0.055 val 0.118 | acc train 0.982 val 0.939 |

```

✓ 5. Curvas de aprendizaje: ResNet-18 vs. EfficientNet-B0

Mismo diagnostico que en 1.4: comparen el gap train/val de cada modelo, y ahora tambien comparenlos *entre sí* — ¿cual generaliza mejor con el mismo presupuesto de datos y epocas?

```

historiales = {
    "ResNet-18": hist_resnet,
    "EfficientNet-B0": hist_efficientnet
}

colores = {
    "ResNet-18": "tab:blue",
    "EfficientNet-B0": "tab:orange"
}

epocas = range(1, N_EPOCAS + 1)

fig, axs = plt.subplots(1, 2, figsize=(13, 4.5))

for nombre, h in historiales.items():

```

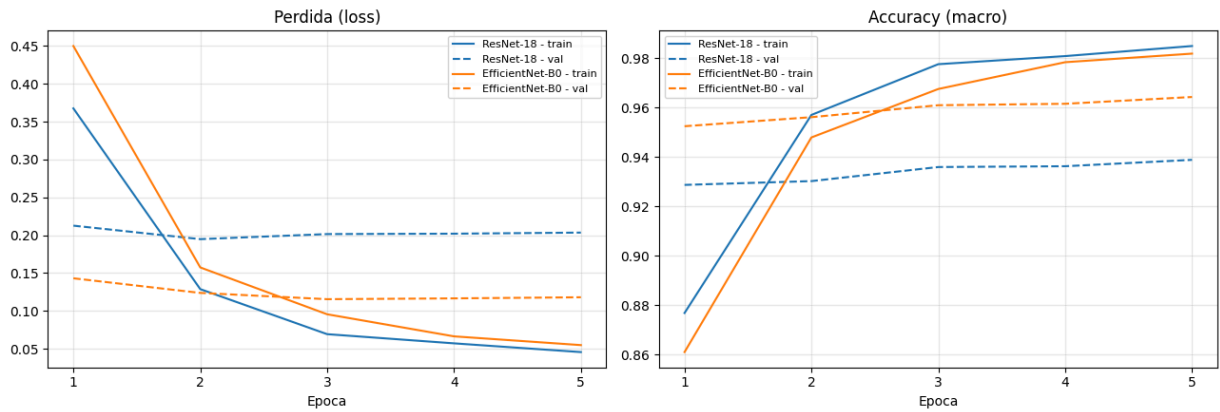
```
for ax, clave in [
    (axs[0], "loss"),
    (axs[1], "accuracy")
]:
    ax.plot(
        epocas,
        [r[clave] for r in h["train"]],
        color=colores[nombre],
        label=f"{nombre} - train"
    )

    ax.plot(
        epocas,
        [r[clave] for r in h["val"]],
        color=colores[nombre],
        linestyle="--",
        label=f"{nombre} - val"
    )

axs[0].set_title("Perdida (loss)")
axs[1].set_title("Accuracy (macro)")

for ax in axs:
    ax.set_xlabel("Epoca")
    ax.set_xticks(list(epocas))
    ax.grid(alpha=0.3)
    ax.legend(fontsize=8)

plt.tight_layout()
plt.show()
```



✓ 6. Reporte final sobre el set de prueba

Evaluamos ambos modelos sobre las 10,000 imagenes de prueba de CIFAR-10

```
def evaluar_en_prueba(modelo, nombre_modelo):
    metrica_acc = MulticlassAccuracy(num_classes=10, average="macro").to(DISPOSITIVO)
    metrica_prec = MulticlassPrecision(num_classes=10, average="macro").to(DISPOSITIVO)
    metrica_rec = MulticlassRecall(num_classes=10, average="macro").to(DISPOSITIVO)
    metrica_f1 = MulticlassF1Score(num_classes=10, average="macro").to(DISPOSITIVO)
    metrica_matriz = MulticlassConfusionMatrix(num_classes=10).to(DISPOSITIVO)

    modelo.eval()
    with torch.no_grad():
        for imagenes, etiquetas in test_loader:
            imagenes, etiquetas = imagenes.to(DISPOSITIVO), etiquetas.to(DISPOSITIVO)
            preds = modelo(imagenes).argmax(dim=1)
            for m in [metrica_acc, metrica_prec, metrica_rec, metrica_f1, metrica_matriz]:
                m.update(preds, etiquetas)

    resumen = {
        "modelo": nombre_modelo,
        "accuracy": metrica_acc.compute().item(),
        "precision": metrica_prec.compute().item(),
```

```

    "recall": metrica_rec.compute().item(),
    "f1": metrica_f1.compute().item(),
}
return resumen, metrica_matriz.compute().cpu().numpy()

```

```

resumen_resnet, matriz_resnet = evaluar_en_prueba(modelo_resnet, "ResNet-18")
resumen_efficientnet, matriz_efficientnet = evaluar_en_prueba(modelo_efficientnet, "EfficientNet-B0")

```

```

df_comparativa = pd.DataFrame([resumen_resnet, resumen_efficientnet]).round(3)
df_comparativa

```

	modelo	accuracy	precision	recall	f1	
0	ResNet-18	0.933	0.934	0.933	0.933	
1	EfficientNet-B0	0.960	0.960	0.960	0.960	

Próximos pasos:

[Generar código con df_comparativa](#)[New interactive sheet](#)

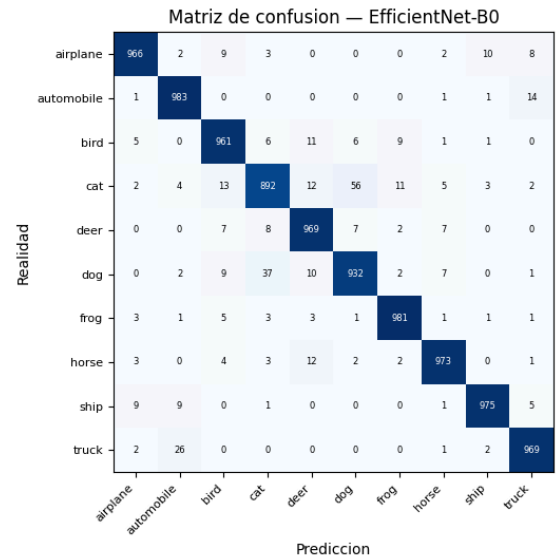
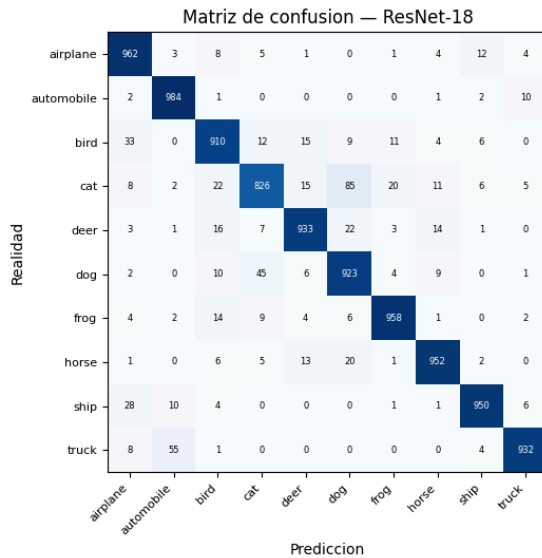
```

fig, axs = plt.subplots(1, 2, figsize=(13, 5.5))

for ax, matriz, nombre in [(axs[0], matriz_resnet, "ResNet-18"), (axs[1], matriz_efficientnet, "EfficientNet-B0")]:
    im = ax.imshow(matriz, cmap="Blues")
    ax.set_xticks(range(10)); ax.set_xticklabels(nombres_clases, rotation=45, fontweight="bold")
    ax.set_yticks(range(10)); ax.set_yticklabels(nombres_clases, fontweight="bold", fontsize=8)
    ax.set_xlabel("Predicción"); ax.set_ylabel("Realidad")
    ax.set_title(f"Matriz de confusion - {nombre}")
    for i in range(10):
        for j in range(10):
            color = "white" if matriz[i, j] > matriz.max() / 2 else "black"
            ax.text(j, i, str(int(matriz[i, j])), ha="center", va="center", color=color)

plt.tight_layout()
plt.show()

```



7. Reporte de diseño (completar en equipo)

Esta seccion es, literalmente, la evidencia "**Reporte de diseño**" del temario de la Unidad 1. Respondan cada punto con base en lo que observaron arriba — pueden editar esta celda de markdown directamente o copiar las respuestas a un documento aparte.

1. Arquitectura y justificación

(¿Cuál de las dos arquitecturas usarían si solo pudieran quedarse con una para este problema? Justifiquen con los números de la Parte 6, no solo con la teoría de 1.3.)

Elegiríamos **EfficientNet-B0**, ya que obtuvo mejores resultados en el conjunto de prueba. EfficientNet-B0 obtuvo una accuracy de **0.960**, mientras que ResNet-18 obtuvo **0.933**. También obtuvo mejores resultados

en precision, recall y F1-score, con **0.960** en las tres métricas frente a valores de aproximadamente **0.933** para ResNet-18.

2. Hiperparámetros usados

Learning rate: **1e-4** · Optimizador: **Adam** · Épocas: **5** · Batch size: **32** · Tamaño del subconjunto de entrenamiento: **40000** · Pesos iniciales: preentrenados (ImageNet)

3. Estrategia de entrenamiento

El entrenamiento se realizó con **GPU (CUDA)**. ResNet-18 tardó aproximadamente **165 segundos por época**, mientras que EfficientNet-B0 tardó aproximadamente **244 segundos por época**. Si tuviéramos que reentrenar con más tiempo disponible, primero aumentaríamos el tamaño del conjunto de datos de entrenamiento para proporcionar más ejemplos a los modelos.

4. Diagnóstico de las curvas (conexión con 1.4)

(¿Alguno de los dos modelos mostró señales de overfitting según el checklist de la diapositiva 21? ¿Cuál generalizó mejor — menor gap train/val, no necesariamente mayor accuracy?)

Ambos modelos mostraron cierta diferencia entre el desempeño de entrenamiento y validación. ResNet-18 terminó con una accuracy de entrenamiento de **0.985** y una accuracy de validación de **0.939**, mientras que EfficientNet-B0 terminó con **0.982** en entrenamiento y **0.962** en validación. El gap entre train y validation fue menor en EfficientNet-B0, aproximadamente **0.020**, frente a aproximadamente **0.046** en ResNet-18. Por lo tanto, EfficientNet-B0 mostró una mejor generalización en este experimento.

5. Capacidad vs. eficiencia (conexión con 1.3)

(Con el número de parámetros de la Parte 2 y el accuracy final de la Parte 6: ¿cuál modelo ofrece mejor relación desempeño/costo computacional en este experimento?)

ResNet-18 cuenta con **11,181,642 parámetros**, mientras que EfficientNet-B0 cuenta con **4,020,358 parámetros**. EfficientNet-B0 obtuvo una accuracy de **0.960**, superior al **0.933** de ResNet-18, utilizando una

cantidad considerablemente menor de parámetros. Por lo tanto, en este experimento EfficientNet-B0 ofrece una mejor relación entre desempeño y cantidad de parámetros. Sin embargo, su tiempo por época fue mayor, aproximadamente **244 segundos**, frente a **165 segundos** de ResNet-18.

6. Errores más frecuentes

(Con las matrices de confusión de la Parte 6: ¿qué par de clases confunde más cada modelo? ¿Tiene sentido esa confusión dado lo que representan esas clases?)

En ambos modelos, el par de clases que presenta mayor confusión es **gato y perro**. En ResNet-18, **102 imágenes de gatos fueron clasificadas como perros**, mientras que **32 imágenes de perros fueron clasificadas como gatos**. En EfficientNet-B0, **56 imágenes de gatos fueron clasificadas como perros** y **37 imágenes de perros fueron clasificadas como gatos**. Esta confusión tiene sentido porque gato y perro son clases visualmente similares en algunas imágenes y pueden compartir características de forma, textura y apariencia.

8. Cierre y entregables del mini proyecto

CONCLUSIÓN